

Statistical Learning 2: Deep Learning

Course Design Package

Course Overview

- **Level:** Advanced undergraduate / Master's, cross-listed.
- **Format:** Lecture-based, 2 sessions/week, 75 minutes each, 14 weeks (28 sessions total).
- **Assessment:** Weekly in-section quizzes; take-home coding projects. No midterm or final exam.
- **Assumed background:** Statistical Learning 1 (or equivalent intro ML/stats course); linear algebra and multivariate calculus; probability and statistical inference; Python programming.
- **Orientation:** Every architecture is introduced through its statistical motivation first (what modeling assumption or estimation problem it solves) before its engineering mechanics.

Explicit Assumptions

- **Number of projects:** You indicated 4–5 take-home coding projects. This design uses 5, spaced to follow each major thematic arc (foundations, CNNs, Transformers, generative models, capstone). Drop the capstone project (Project 5) to reduce to 4.
- **Quiz schedule:** Weekly in-section quizzes are assumed to run in Weeks 2–14 (13 quizzes total), each covering the immediately preceding week's lecture content. Week 1 has no quiz (no prior content yet); the Week 14 buffer session has no quiz.
- **Grade weighting:** No specific weighting was given, so none is assumed or assigned here. The Assessment Plan below specifies type, timing, and rationale only.
- **Session 28 (last meeting):** Reserved as the buffer/flex session discussed during intake – used here for course synthesis, with the frontier-topics content in Module 14 movable/droppable if the semester needs to compress to 13 weeks.

1 Course Structure

Wk	Sessions	Module	Goal
1	1–2	M1: Neural Networks as Statistical Models	Frame feedforward networks as nonlinear statistical models and connect loss functions to maximum likelihood.
2	3–4	M2: Optimization for Deep Learning	Treat gradient-based training as stochastic approximation and characterize loss-landscape geometry.

Wk	Sessions	Module	Goal
3	5–6	M3: Generalization in the Overparameterized Regime	Reconcile classical bias–variance theory with double descent and implicit regularization.
4	7–8	M4: Regularization & Normalization	Interpret weight decay, dropout, and normalization as statistical devices (priors, model averaging, moment control).
5	9–10	M5: Convolutional Networks & Inductive Bias	Show convolution as an encoded statistical assumption (stationarity, locality) and trace CNN architecture evolution.
6	11–12	M6: Sequential Data – RNNs and Their Limits	Model sequences via recurrence as nonlinear state-space models; diagnose vanishing/exploding gradients.
7	13–14	M7: Attention Mechanisms	Derive attention as a kernel-smoothing / nonparametric-regression operation.
8	15–16	M8: The Transformer Architecture	Assemble the full Transformer block and justify its design choices statistically.
9	17–18	M9: LLMs – Pretraining, Scaling, In-Context Learning	Treat pretraining as large-scale MLE; examine empirical scaling laws and theories of in-context learning.
10	19–20	M10: Latent-Variable Generative Models	Formalize generation via variational inference (VAEs) and exact-likelihood change-of-variables (flows).
11	21–22	M11: Adversarial & Score-Based Generative Models	Contrast implicit (GAN) and score-based (diffusion) generative modeling as different statistical estimation strategies.
12	23–24	M12: Self-Supervised & Contrastive Representation Learning	Frame label-free pretraining objectives as mutual-information / pseudolikelihood estimation problems.
13	25–26	M13: Graph Neural Networks & Structured Data	Extend inductive-bias reasoning to graph-structured data via message passing and spectral methods.
14	27–28	M14: Uncertainty, Frontiers & Synthesis	Cover Bayesian deep learning / calibration, touch modern efficient architectures, and synthesize the course. Session 28 is the buffer session.

2 Prerequisite Map

Topic	Prerequisite(s)	Where Taught
1.1 GLMs → Neural Networks	MLE; linear/logistic regression; log-likelihood gradients	Assumed (Stat. Learning 1); Assumed (Probability & Inference)
1.2 Feedforward Nets, UAT, Backprop	Multivariate chain rule, Jacobians; statistical framing of 1.1	Assumed (Linear Algebra); Module 1, Topic 1.1

Topic	Prerequisite(s)	Where Taught
2.1 SGD & Stochastic Approximation	Gradient descent basics; computing gradients via backprop	Assumed (Stat. Learning 1); Module 1, Topic 1.2
2.2 Adaptive Methods & Loss Landscape	SGD mechanics; eigenvalues/Hessian intuition	Module 2, Topic 2.1; Assumed (Linear Algebra)
3.1 Classical Generalization Theory	Bias-variance tradeoff; concentration inequalities	Assumed (Stat. Learning 1); Assumed (Probability & Inference)
3.2 Double Descent & Implicit Regularization	Classical theory (for contrast); optimization dynamics	Module 3, Topic 3.1; Module 2
4.1 Weight Decay, Dropout, Bayesian View	MAP estimation, priors; regularized loss functions	Assumed (Probability & Inference); Assumed (Stat. Learning 1)
4.2 Batch/Layer Normalization	Optimization dynamics; sample mean/variance estimators	Module 2; Assumed (Probability & Inference)
5.1 Convolution as Inductive Bias	Linear algebra of weight matrices; feedforward layer mechanics	Assumed (Linear Algebra); Module 1, Topic 1.2
5.2 Modern CNN Architectures	Convolution basics; backprop through deep stacks	Module 5, Topic 5.1; Module 1 & Module 2
6.1 RNNs as State-Space Models	Sequential/time-series intuition; computational graphs	Assumed (Stat. Learning 1); Module 1, Topic 1.2
6.2 LSTM/GRU & Vanishing Gradients	RNN mechanics, BPTT; chain rule over many steps	Module 6, Topic 6.1; Module 1, Topic 1.2
7.1 Attention as Kernel Regression	Nonparametric regression / kernel smoothing; softmax	Assumed (Stat. Learning 1); Assumed (Probability & Inference)
7.2 Self-Attention & Multi-Head	Attention basics; matrix projections	Module 7, Topic 7.1; Assumed (Linear Algebra)
8.1 Full Transformer Block	Self-attention; normalization	Module 7, Topic 7.2; Module 4, Topic 4.2
8.2 Encoder-Decoder vs. Decoder-Only	Transformer block; conditional probability modeling	Module 8, Topic 8.1; Assumed (Probability & Inference)
9.1 Pretraining Objectives & Scaling Laws	Autoregressive MLE; Transformer architecture	Module 1, Topic 1.1; Module 8
9.2 In-Context Learning & Fine-Tuning	Pretraining objectives; Bayesian inference intuition	Module 9, Topic 9.1; Assumed (Probability & Inference)
10.1 Variational Autoencoders	Bayesian inference, KL divergence; latent-variable MLE	Assumed (Probability & Inference); Module 1, Topic 1.1
10.2 Normalizing Flows	Latent-variable framing; Jacobians / change of variables	Module 10, Topic 10.1; Assumed (Linear Algebra)
11.1 Generative Adversarial Networks	Generative modeling framing; minimax games (new)	Module 10, Topic 10.1; introduced in class
11.2 Diffusion & Score-Based Models	Latent-variable/generative framing; stochastic processes (new)	Module 10; Assumed (Probability), supplemented in class

Topic	Prerequisite(s)	Where Taught
12.1 Contrastive Learning	Encoder representations; mutual information / KL	Module 5 & Module 8; Assumed (Probability & Inference)
12.2 Masked & Predictive Self-Supervision	Pretraining objectives; contrastive framing	Module 9, Topic 9.1; Module 12, Topic 12.1
13.1 Message Passing / GNNs	Adjacency matrices; feedforward/aggregation mechanics	Assumed (Linear Algebra); Module 1
13.2 Spectral GNNs & Expressiveness	Message passing; eigen-decomposition	Module 13, Topic 13.1; Assumed (Linear Algebra)
14.1 Bayesian Deep Learning & UQ	Bayesian inference; dropout/ensembling	Assumed (Probability & Inference); Module 4, Topic 4.1
14.2 Frontiers & Synthesis	Cumulative – entire course	Modules 1–13

3 Assessment Plan

Module(s)	Assessment	Timing	Rationale
All (recurring)	In-section quiz	Weeks 2–14, weekly	Frequent low-stakes checks maintain pace and catch misconceptions early since there is no midterm/final to force review.
M1–M3	Project 1: Foundations – build an MLP and optimizer from scratch; empirically reproduce a double-descent curve	Due end of Week 3	Tests whether students can translate statistical theory (MLE framing, bias–variance, double descent) into working, measurable code.
M5	Project 2: CNN image classifier – implement and ablate a small ResNet	Due end of Week 5	Tests understanding of convolution as inductive bias and the statistical effect of architectural choices (depth, skip connections).
M7–M9	Project 3: Attention/Transformer from scratch, or fine-tune a small pretrained LM	Due end of Week 9	Tests mastery of self-attention mechanics and the pretrain/fine-tune statistical objective distinction.
M10–M11	Project 4: Generative modeling comparison – implement a VAE and a simple diffusion model on the same dataset	Due end of Week 12	Tests ability to compare likelihood-based vs. score-based generative estimation empirically, not just describe it.

Module(s)	Assessment	Timing	Rationale
M12–M14	Project 5 (capstone): Self-supervised or GNN mini-project of the student’s choosing, framed in statistical-estimation terms	Due end of Week 14	Synthesizes the course’s statistical lens on a modern architecture without prescribing a single “correct” pipeline.

4 Module 1: Neural Networks as Statistical Models

4.1 Topic 1.1: From GLMs to Neural Networks

Core Concepts:

- Neural networks as compositions of generalized linear models (GLMs): each layer is an affine map followed by a link-like nonlinearity.
- Loss functions as negative log-likelihoods under an assumed output-noise model: squared error $\leftrightarrow$ Gaussian noise, cross-entropy $\leftrightarrow$ Bernoulli/categorical noise.
- Choice of final-layer activation (identity, sigmoid, softmax) as a choice of exponential-family output distribution.
- Maximum likelihood estimation: $\hat{\theta} = \arg \max_{\theta} \sum_i \log p_{\theta}(y_i | x_i)$.

Learning Objectives:

- Explain why minimizing squared error is equivalent to Gaussian MLE.
- Derive the cross-entropy loss from the Bernoulli/categorical log-likelihood.
- Apply the GLM-to-network analogy to justify an output layer's activation function given a stated noise model.

Example Questions:

1. (*Basic*) Write the negative log-likelihood for $y_i | x_i \sim \mathcal{N}(f_{\theta}(x_i), \sigma^2)$ and show it reduces to squared error up to an additive constant and a scale factor.
Answer: $-\log p = \frac{1}{2\sigma^2}(y_i - f_{\theta}(x_i))^2 + \text{const}$; minimizing over θ is equivalent to minimizing squared error.
2. (*Intermediate*) Derive the binary cross-entropy loss from the Bernoulli likelihood with $p_{\theta}(y = 1 | x) = \sigma(f_{\theta}(x))$.
Rubric: correctly writes $p^y(1-p)^{1-y}$, takes $-\log$, substitutes the sigmoid; full credit requires stating this is exact MLE, not an approximation.
3. (*Advanced*) A student proposes using squared error to train a classifier whose targets are one-hot class labels. Statistically, what noise/output-distribution assumption does this implicitly impose, and why is it usually a poor match for classification?
Rubric: identifies the implicit Gaussian-noise assumption on a categorical target, notes miscalibrated gradients / poor probability estimates near saturation as the consequence.

4.2 Topic 1.2: Feedforward Networks, Universal Approximation, and Back-propagation

Core Concepts:

- Universal approximation theorem: a single hidden layer with enough units can approximate any continuous function on a compact domain – a statement about expressivity, not learnability.
- Computational graphs and reverse-mode automatic differentiation as an efficient application of the multivariate chain rule.
- Layer-wise gradient recursion: $\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial W^{(l)}}$, with $\partial L / \partial a^{(l)}$ propagated backward from the output.
- Expressivity vs. trainability vs. generalization as three distinct questions (this topic addresses only the first).

Learning Objectives:

- Explain the universal approximation theorem's claim and its limits (it says nothing about optimization or sample efficiency).

- Derive the backpropagation recursion for a two-layer network by hand.
- Apply automatic differentiation reasoning to trace a gradient through a small computational graph with a branching (skip) connection.

Common Difficulty & Mitigation: Students often experience backprop as “notational overload” – losing track of which Jacobian multiplies which, especially with branching graphs. Mitigation: require a from-scratch backprop derivation for a 2-layer net by hand *before* showing the general computational-graph formalism, then re-derive the same result using the graph to show it is the same chain rule, just organized.

Example Questions:

1. (*Basic*) For $f(x) = W_2\sigma(W_1x + b_1) + b_2$, write $\partial L/\partial W_1$ in terms of $\partial L/\partial f$.
Answer: standard 2-layer backprop chain, checking correct transpose placement.
2. (*Intermediate*) A node in a computational graph feeds into two downstream paths. State the rule for combining the two upstream gradients at that node and explain why it is a direct consequence of the multivariate chain rule.
Rubric: correctly states gradients sum (not multiply) at a fan-out node; ties this to partial derivatives of a function of multiple paths.
3. (*Advanced*) Explain why the universal approximation theorem does not imply that gradient descent will find a good approximator in practice.
Rubric: distinguishes existence of weights from a learning algorithm finding them; may mention nonconvexity, sample complexity, or optimization landscape as the missing piece.

5 Module 2: Optimization for Deep Learning

5.1 Topic 2.1: SGD and Stochastic Approximation

Core Concepts:

- SGD update: $\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} L(\theta_t; x_{it}, y_{it})$, an unbiased noisy estimate of the full-batch gradient.
- Robbins–Monro conditions for convergence of stochastic approximation: $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$.
- Minibatch gradient noise as an implicit regularizer – connects optimization choice to generalization (previewed here, developed fully in Module 3).
- Learning-rate schedules (step decay, cosine, warmup) as controlling the noise/signal tradeoff over training.

Learning Objectives:

- Explain why minibatch gradients are unbiased but high-variance estimators of the full gradient.
- Apply the Robbins–Monro conditions to evaluate whether a given learning-rate schedule is theoretically guaranteed to converge (in the convex case).
- Evaluate the tradeoff between batch size, gradient noise, and wall-clock training efficiency.

Example Questions:

1. (*Basic*) Show that the minibatch gradient is an unbiased estimator of the full-batch gradient under uniform sampling without replacement (approximately) or with replacement (exactly).
Answer: linearity of expectation over the sampled indices.
2. (*Intermediate*) Does the constant learning rate $\eta_t = \eta$ satisfy the Robbins–Monro conditions? What does this imply about SGD’s limiting behavior with constant step size?
Rubric: correctly shows $\sum \eta_t^2$ diverges too, so SGD does not converge to a point but to a stationary distribution around the optimum.

3. (*Advanced*) A colleague claims “larger batch sizes are strictly better because the gradient estimate has lower variance.” Give a statistical and an empirical-generalization argument against this claim.

Rubric: statistical argument re: diminishing returns ($\text{Var} \propto 1/B$ but compute grows linearly with B); generalization argument re: noise-as-regularizer connecting to flatter minima.

5.2 Topic 2.2: Adaptive Methods and Loss Landscape Geometry

Core Concepts:

- Momentum and adaptive per-parameter learning rates (RMSProp, Adam) as statistics of past gradients (running means/variances).
- Adam update sketch: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, $\theta_{t+1} = \theta_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$.
- Loss landscape geometry: saddle points dominate over local minima in high dimensions; sharp vs. flat minima and their empirical link to generalization.
- Hessian eigenvalue spectrum as a diagnostic of curvature and conditioning.

Learning Objectives:

- Explain how Adam’s per-parameter scaling addresses ill-conditioned curvature differently than plain SGD.
- Derive why, in high dimensions, saddle points vastly outnumber local minima under a random-matrix-theory argument (informally).
- Evaluate empirical evidence linking flat minima to better generalization, including its limitations.

Example Questions:

1. (*Basic*) What statistical quantities do m_t and v_t in Adam estimate, and why is bias correction ($\hat{m}_t$, $\hat{v}_t$) needed early in training?

Answer: exponential moving averages of first/second gradient moments; early estimates are biased toward zero because $m_0 = v_0 = 0$.

2. (*Intermediate*) Explain, in terms of the Hessian’s eigenvalue spectrum, why plain SGD converges slowly on an ill-conditioned quadratic and why Adam-style scaling can help.

Rubric: references condition number (ratio of largest to smallest eigenvalue) and per-coordinate rescaling.

3. (*Advanced*) Sharp minima and flat minima can achieve identical training loss. Give a statistical argument for why flat minima might generalize better, and one documented critique of this claim.

Rubric: argument re: robustness to parameter/test-time perturbation or description-length intuition; critique re: sharpness measures being reparameterization-dependent (Dinh et al.-style critique), accepted without requiring the citation by name.

6 Module 3: Generalization in the Overparameterized Regime

6.1 Topic 3.1: Classical Generalization Theory Revisited

Core Concepts:

- Bias–variance decomposition as the classical lens: test error = bias² + variance + irreducible noise.
- VC dimension and Rademacher complexity as classical capacity measures bounding the generalization gap.

- Why these classical bounds are vacuous for deep networks: parameter counts vastly exceed sample sizes, yet test error remains low.

Learning Objectives:

- Explain the classical bias–variance tradeoff and the U-shaped risk curve it predicts as a function of model capacity.
- Evaluate why VC-dimension-based generalization bounds become numerically vacuous (bound $\gg 1$) for typical deep networks.
- Formulate, in your own words, the puzzle that motivates Topic 3.2: why do overparameterized networks generalize at all?

Example Questions:

1. (*Basic*) Sketch the classical U-shaped test-error-vs-capacity curve and label the underfitting and overfitting regions.
Answer: standard bias-dominated / variance-dominated regions with a minimum at intermediate capacity.
2. (*Intermediate*) A VC-dimension bound for a network with 10^7 parameters and $n = 5 \times 10^4$ training samples gives a generalization-gap bound greater than 1. Explain, numerically, why this makes the bound useless even though the network generalizes well in practice.
Rubric: recognizes the bound exceeds the maximum possible error (since error $\in [0, 1]$ for 0-1 loss), hence provides no information.
3. (*Advanced*) Why can't the bias–variance decomposition, taken at face value, explain a network that has zero training error (zero bias) yet still generalizes well (should predict high variance)?
Rubric: identifies the apparent contradiction as the seed of the double-descent discussion; no full resolution expected here.

6.2 Topic 3.2: Double Descent and Implicit Regularization

Core Concepts:

- The double-descent curve: test error rises then falls again as model capacity crosses the interpolation threshold, contradicting the classical U-shape.
- Implicit regularization of SGD: among the many zero-training-error solutions, gradient descent is biased toward low-norm / flat / “simple” ones.
- Neural Tangent Kernel (NTK) intuition: in the infinite-width limit, network training approximates kernel regression with a fixed kernel, giving one theoretical handle on why overparameterization can help.

Learning Objectives:

- Explain the double-descent phenomenon and identify the interpolation threshold on a plotted curve.
- Derive (at a sketch level) why gradient descent from small initialization is biased toward minimum-norm interpolating solutions in an overparameterized linear model.
- Evaluate the NTK view's explanatory power and its known limitations (it describes a lazy-training regime that real, feature-learning networks often leave).

Common Difficulty & Mitigation: Double descent directly contradicts the bias–variance intuition students just built in 3.1, which is precisely the point – but it is a common misconception source if presented too quickly. Mitigation: have students first empirically reproduce a double-descent curve on a small model (tied to Project 1) before presenting the NTK/implicit-regularization explanation, so the phenomenon is concrete before the theory arrives.

Example Questions:

1. (*Basic*) On a double-descent plot, label the classical (underparameterized) regime, the interpolation threshold, and the overparameterized regime.
Answer: standard three-region labeling with the interpolation-threshold spike in test error.
2. (*Intermediate*) For an overparameterized linear regression solved by gradient descent from zero initialization, explain informally why the solution found is the minimum- ℓ_2 -norm interpolant among all zero-training-error solutions.
Rubric: references gradient descent staying in the row space of the design matrix; full derivation not required, correct informal mechanism is.
3. (*Advanced*) The NTK regime predicts that infinite-width networks behave like fixed-kernel regression. Explain why this is in tension with the empirical success of representation/feature learning in real (finite-width) deep networks.
Rubric: identifies that NTK/lazy training implies features do *not* change much during training, whereas empirically useful representations clearly do evolve; a strong answer notes this is an active research tension, not fully resolved.

7 Module 4: Regularization and Normalization

7.1 Topic 4.1: Weight Decay, Dropout, and Bayesian Interpretations

Core Concepts:

- Weight decay as an ℓ_2 penalty, equivalent to MAP estimation under a Gaussian prior on parameters: $\arg \max_{\theta} \log p(\mathcal{D} \mid \theta) + \log p(\theta)$.
- Dropout: $\tilde{h} = h \odot m$, $m_i \sim \text{Bernoulli}(p)$, as training an implicit ensemble of subnetworks and, at test time, an approximation to averaging over that ensemble.
- Dropout's connection to approximate Bayesian inference (MC-dropout as an approximate posterior predictive), previewed for Module 14.

Learning Objectives:

- Derive the equivalence between ℓ_2 -regularized MLE and MAP estimation under a Gaussian prior.
- Explain dropout as approximate model averaging over an exponential family of subnetworks.
- Evaluate when weight decay and dropout are redundant vs. complementary, given what statistical assumption each encodes.

Common Difficulty & Mitigation: The MAP/Bayesian-prior reading of weight decay is often the first time students see regularization reinterpreted probabilistically rather than as an ad hoc penalty; the algebra is short but the conceptual reframe is not obvious. Mitigation: work the Gaussian-prior MAP derivation on the board end-to-end with explicit correspondence between λ and prior variance, then have students independently derive the analogous Laplace-prior $\leftrightarrow \ell_1$ correspondence as a check.

Example Questions:

1. (*Basic*) Starting from a Gaussian prior $\theta \sim \mathcal{N}(0, \tau^2 I)$, derive the MAP objective and show it equals the ℓ_2 -regularized log-likelihood. State how λ relates to τ^2 and the noise variance.
Answer: $\lambda \propto \sigma^2 / \tau^2$ (up to the specific noise model used).
2. (*Intermediate*) Explain why dropout at training time but scaling activations by p at test time (inverted dropout) approximates ensemble averaging.
Rubric: references averaging over the 2^n possible subnetworks and the expectation-matching argument for the test-time scale factor.
3. (*Advanced*) If a model already uses strong weight decay, is additional dropout still statistically justified? Under what data/architecture conditions would you expect the two to be redundant

vs. complementary?

Rubric: no single correct answer required; credit for correctly identifying that weight decay controls parameter magnitude while dropout controls co-adaptation/redundancy among units – different statistical mechanisms.

7.2 Topic 4.2: Batch and Layer Normalization

Core Concepts:

- BatchNorm: $\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, using minibatch sample statistics as a plug-in estimator of activation mean/variance, followed by a learned affine transform.
- LayerNorm as the analogous per-example (rather than per-batch) normalization, removing dependence on batch composition – important for sequence models (Module 6 onward).
- The “internal covariate shift” explanation vs. the competing loss-landscape-smoothing explanation for why normalization helps optimization.

Learning Objectives:

- Explain the statistical operation BatchNorm performs and why it is a biased estimator at small batch sizes.
- Contrast BatchNorm and LayerNorm in terms of which axis the summary statistics are computed over.
- Evaluate the empirical evidence for and against the internal-covariate-shift explanation of why normalization aids training.

Example Questions:

1. (*Basic*) Write the BatchNorm forward transform for a single activation and identify which statistics are estimated from the minibatch versus which are learned parameters.
Answer: μ_B, σ_B^2 estimated from batch; γ, β learned.
2. (*Intermediate*) Why does BatchNorm behave poorly with very small batch sizes, and why is LayerNorm preferred for Transformers regardless of batch size?
Rubric: small-batch BatchNorm has high-variance statistics estimates; LayerNorm normalizes per example, so it is invariant to batch composition and works with variable-length sequences.
3. (*Advanced*) Summarize the empirical case (Santurkar et al.-style argument, without requiring the citation) that BatchNorm’s benefit comes from smoothing the loss landscape rather than reducing internal covariate shift, and describe one experiment that would distinguish the two explanations.
Rubric: credit for proposing any reasonable ablation (e.g., artificially injecting covariate shift after BatchNorm and checking if training still improves).

8 Module 5: Convolutional Networks and Inductive Bias

8.1 Topic 5.1: Convolution as Statistical Inductive Bias

Core Concepts:

- Convolution operation: $(f * g)(t) = \sum_{\tau} f(\tau)g(t - \tau)$; weight sharing across spatial locations as a hard statistical assumption of translation-equivariance/stationarity.
- Weight sharing as variance reduction: drastically fewer free parameters than a fully-connected layer, at the cost of introducing bias if the stationarity assumption is wrong.
- Receptive fields and pooling as forms of statistical aggregation (local averaging/max as a summary statistic) that build spatial invariance.

Learning Objectives:

- Explain convolution as encoding a translation-equivariance assumption, in bias–variance terms.
- Derive how receptive field size grows with network depth for a stack of $k \times k$ convolutions.
- Evaluate when the stationarity assumption behind convolution is a good vs. poor match for a given data domain.

Example Questions:

1. (*Basic*) For a stack of 3 convolutional layers each with a 3×3 kernel and stride 1, compute the receptive field size at the output.
Answer: 7×7 (each layer adds $(k - 1)$ to each side, cumulatively).
2. (*Intermediate*) In bias–variance terms, explain what is gained and what is potentially lost by replacing a fully-connected layer with a convolutional layer of equivalent output size.
Rubric: variance reduction from far fewer parameters vs. bias introduced if long-range or non-local dependencies matter.
3. (*Advanced*) Give a data domain where the translation-stationarity assumption behind standard convolution is a poor fit, and describe (at a conceptual level) an architectural modification that relaxes it.
Rubric: any reasonable example (e.g., images with strong positional structure like faces, where fully-local weight sharing discards useful position information) paired with a plausible fix (e.g., locally-connected layers, position-dependent modulation).

8.2 Topic 5.2: Modern CNN Architectures

Core Concepts:

- Depth vs. trainability: very deep plain CNNs suffer degradation even on training error, motivating residual connections.
- Residual connections as statistical residual modeling: $h^{(l+1)} = h^{(l)} + F(h^{(l)})$, learning a correction to the identity rather than an arbitrary new mapping.
- Brief survey of architectural motifs (Inception-style multi-scale filters, depthwise-separable convolutions) as different bias–variance trades within the convolutional family.

Learning Objectives:

- Explain why residual connections address the degradation problem, distinct from the vanishing-gradient explanation alone.
- Apply the residual-modeling framing to predict what a residual block learns when the optimal mapping is close to identity.
- Compare at least two architectural motifs (e.g., residual vs. Inception-style) in terms of what statistical structure they exploit.

Example Questions:

1. (*Basic*) Write the residual block equation and explain why it is easier to optimize toward the identity function than a plain (non-residual) block.
Answer: $h^{(l+1)} = h^{(l)} + F(h^{(l)})$; identity is achieved simply by driving $F \rightarrow 0$, versus needing to learn an exact identity mapping through nonlinear layers.
2. (*Intermediate*) The “degradation problem” in very deep plain networks is a training-error phenomenon, not just a generalization one. Explain why this rules out overfitting/variance as the explanation and points instead toward optimization difficulty.
Rubric: correctly separates training-error degradation (optimization issue) from test-error degradation (generalization issue).
3. (*Advanced*) Compare residual connections and Inception-style multi-branch filters in terms of what statistical structure in the data each is designed to exploit.

Rubric: residual connections target optimization/depth; Inception-style motifs target multi-scale spatial structure. Partial credit for either point made clearly.

9 Module 6: Sequential Data – RNNs and Their Limits

9.1 Topic 6.1: RNNs as Nonlinear State-Space Models

Core Concepts:

- Recurrence relation: $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$ – a nonlinear analogue of a linear state-space model, with h_t acting as a (learned, not necessarily sufficient) statistic summarizing the past.
- Backpropagation through time (BPTT): unrolling the recurrence into a deep computational graph and applying standard backprop.
- Parameter sharing across time steps as another instance of the inductive-bias-via-weight-sharing theme from Module 5.

Learning Objectives:

- Explain the analogy and disanalogy between RNN hidden states and sufficient statistics in classical state-space models.
- Derive the BPTT gradient recursion for a simple RNN by unrolling 3 time steps.
- Apply the weight-sharing-as-inductive-bias framing from Module 5 to explain why RNN parameters are shared across time.

Example Questions:

1. (*Basic*) Unroll a simple RNN for $t = 1, 2, 3$ and write the resulting computational graph.
Answer: chain of three recurrence applications sharing W_h, W_x, b .
2. (*Intermediate*) In what sense is an RNN hidden state analogous to a sufficient statistic in a classical (linear-Gaussian) state-space model, and in what sense is the analogy imperfect?
Rubric: analogy: both aim to compress history into a fixed-size summary sufficient for prediction; imperfect because there is no guarantee of statistical sufficiency for a learned, nonlinear h_t .
3. (*Advanced*) Explain why sharing W_h, W_x across all time steps is a stronger inductive-bias assumption than sharing convolutional weights across spatial locations, in terms of what invariance each assumes.
Rubric: credit for identifying time-homogeneity/stationarity of the dynamics as the RNN's implicit assumption, analogous to but distinct from spatial stationarity in CNNs.

9.2 Topic 6.2: LSTM/GRU and the Vanishing/Exploding Gradient Problem

Core Concepts:

- Vanishing/exploding gradients in vanilla RNNs: repeated multiplication by W_h (or its Jacobian) over many time steps causes gradient magnitude to shrink or blow up exponentially.
- Gating mechanisms (LSTM forget/input/output gates, GRU update/reset gates) as learned, data-dependent statistics controlling how much of the past state to retain vs. overwrite.
- The additive (rather than purely multiplicative) cell-state update in LSTMs as the specific mechanism that preserves gradient flow.

Learning Objectives:

- Explain, via the repeated-Jacobian-multiplication argument, why vanilla RNN gradients vanish or explode over long sequences.
- Derive how the LSTM's additive cell-state update avoids the same repeated-multiplication failure mode.

- Evaluate when a GRU’s simpler gating is an adequate substitute for an LSTM’s three-gate structure.

Common Difficulty & Mitigation: The gating equations (input, forget, output for LSTM; update, reset for GRU) create notational overload that can obscure the single unifying idea – gates are just learned, sigmoid-valued statistics that control an additive information flow. Mitigation: derive the vanishing-gradient problem for a vanilla RNN first (concrete, small), then introduce the LSTM cell state purely as “what if we made this update additive instead of multiplicative,” and only afterward attach the gating equations as refinements of that one idea.

Example Questions:

1. (*Basic*) For a vanilla RNN, show that the gradient of the loss with respect to h_1 involves a product of $T - 1$ Jacobian terms when backpropagating from h_T , and explain why this causes vanishing or exploding gradients depending on the spectral radius of W_h .
Answer: product $\prod_{t=2}^T \partial h_t / \partial h_{t-1}$; magnitude shrinks/grows exponentially in T depending on whether the dominant eigenvalue is < 1 or > 1 .
2. (*Intermediate*) Explain how the LSTM cell-state update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ avoids the repeated-multiplication problem that plagues vanilla RNN hidden states.
Rubric: identifies the additive term $i_t \odot \tilde{c}_t$ as providing a gradient path that does not require multiplying through many Jacobians, provided $f_t \approx 1$.
3. (*Advanced*) A GRU has fewer parameters and gates than an LSTM. Under what statistical circumstances (sequence length, data regime) would you expect this simplification to cost little, and when would it likely hurt?
Rubric: no single correct answer; credit for reasoning about shorter dependency structure favoring GRU parity with LSTM, and very long-range dependency tasks favoring LSTM’s extra gate.

10 Module 7: Attention Mechanisms

10.1 Topic 7.1: Attention as Kernel Regression

Core Concepts:

- Nadaraya–Watson kernel regression as the statistical ancestor of attention:

$$\hat{y}(x) = \frac{\sum_i K(x, x_i) y_i}{\sum_i K(x, x_i)}.$$

- Attention as a learned, data-dependent kernel: queries and keys replace a fixed kernel bandwidth/shape with a similarity function the model learns.
- Softmax as normalizing similarity scores into convex-combination weights – directly analogous to normalizing kernel weights in Nadaraya–Watson.

Learning Objectives:

- Explain the correspondence between Nadaraya–Watson kernel regression and single-head attention, term by term.
- Derive the softmax-attention weighted average from the general kernel-smoother formula.
- Evaluate what attention gains by learning the similarity function rather than fixing it (e.g., a Gaussian kernel).

Common Difficulty & Mitigation: Students often see attention purely as an engineering trick (“a lookup over vectors”) and miss that it is a direct generalization of a classical nonparametric regression estimator they may already know. Mitigation: present Nadaraya–Watson first with a

fixed Gaussian kernel and small worked numeric example, then substitute a learned similarity function for the kernel and show the formula for query/key/value attention drops out directly.

Example Questions:

1. (*Basic*) Write the Nadaraya–Watson estimator with a Gaussian kernel and identify which term in single-head attention plays the role of $K(x, x_i)$.
Answer: $\exp(-\|x - x_i\|^2 / 2h^2)$ corresponds to the unnormalized attention score, e.g. $\exp(q^\top k_i / \sqrt{d_k})$.
2. (*Intermediate*) Explain why the softmax normalization in attention plays exactly the same statistical role as the denominator $\sum_i K(x, x_i)$ in Nadaraya–Watson.
Rubric: both convert unnormalized similarity/kernel weights into a valid convex combination (weights summing to 1).
3. (*Advanced*) Nadaraya–Watson uses a fixed kernel bandwidth h ; attention effectively learns a data-dependent notion of “bandwidth” via the query/key projections. Explain, statistically, why a learned similarity function can outperform a fixed kernel on structured data like language.
Rubric: credit for noting that a fixed-shape kernel imposes one global notion of similarity, while learned projections can adapt similarity per-task/per-layer to task-relevant structure.

10.2 Topic 7.2: Self-Attention and Multi-Head Attention

Core Concepts:

- Self-attention: $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) V$, with Q, K, V all linear projections of the same input sequence.
- Scaling by $\sqrt{d_k}$ as variance control: keeps dot-product logits from growing with dimension and saturating the softmax.
- Multi-head attention as running several independent learned similarity functions in parallel subspaces, then combining – a statistical ensembling-within-a-layer argument.
- Computational complexity: $O(n^2d)$ in sequence length n , a key practical constraint carried into Module 8/9.

Learning Objectives:

- Derive why scaling by $\sqrt{d_k}$ is necessary by analyzing the variance of $q^\top k$ under random initialization.
- Explain multi-head attention as a statistical ensembling mechanism operating within a single layer.
- Evaluate the quadratic complexity of self-attention and its practical implications for sequence length.

Example Questions:

1. (*Basic*) If $q, k \in \mathbb{R}^{d_k}$ have i.i.d. zero-mean, unit-variance entries, show that $\text{Var}(q^\top k) = d_k$, motivating the $1/\sqrt{d_k}$ scaling.
Answer: sum of d_k i.i.d. products, each with variance 1, gives total variance d_k ; dividing by $\sqrt{d_k}$ restores unit variance.
2. (*Intermediate*) Explain why multiple attention heads with smaller per-head dimension can be viewed as a statistical ensembling technique, analogous in spirit to averaging multiple weak estimators.
Rubric: each head learns a different similarity subspace/pattern; combining reduces the risk of any one head’s idiosyncratic errors dominating.
3. (*Advanced*) Self-attention’s $O(n^2d)$ cost makes long sequences expensive. Propose, at a conceptual level, one statistical approximation (e.g., sparsifying, low-rank, or sampling the attention matrix) that could reduce this cost, and state what statistical assumption it relies on.

Rubric: credit for any reasonable approximation strategy paired with the correct underlying assumption (e.g., low-rank approximation assumes the attention matrix has low effective rank).

11 Module 8: The Transformer Architecture

11.1 Topic 8.1: The Full Transformer Block

Core Concepts:

- Assembling a full block: multi-head self-attention, residual connection, LayerNorm, position-wise feedforward network, second residual connection and LayerNorm.
- Positional encodings: since self-attention is permutation-equivariant, position must be injected explicitly (sinusoidal encodings as a fixed statistical basis; learned positional embeddings as an alternative).
- Pre-LN vs. post-LN placement and its effect on gradient flow/training stability at depth.

Learning Objectives:

- Explain why self-attention alone is permutation-equivariant and why this necessitates explicit positional information.
- Derive the sinusoidal positional encoding's key property (relative positions expressible as a linear function of absolute encodings) at a sketch level.
- Evaluate the tradeoff between pre-LN and post-LN block designs in terms of training stability at depth.

Common Difficulty & Mitigation: Students frequently treat the Transformer block as an unmotivated list of components to memorize. Mitigation: build the block incrementally on the board, motivating each addition by a specific failure mode it fixes (attention alone $\rightarrow$ no position information $\rightarrow$ add positional encoding; deep stacks $\rightarrow$ optimization difficulty $\rightarrow$ add residual+norm), rather than presenting the full diagram at once.

Example Questions:

1. (*Basic*) Show that permuting the input tokens to a self-attention layer permutes the output identically (i.e., self-attention is permutation-equivariant), and explain why this means position must be injected separately.
Answer: attention weights depend only on pairwise content similarity, not index, so reordering inputs reorders outputs correspondingly with no other change.
2. (*Intermediate*) Explain the claimed benefit of sinusoidal positional encodings – that the encoding of position $p + k$ can be written as a linear function of the encoding of position p – and why this might help the model generalize to relative positions.
Rubric: correctly invokes the trigonometric addition identities; full derivation not required, correct qualitative mechanism is.
3. (*Advanced*) Compare pre-LN and post-LN Transformer blocks in terms of gradient flow at large depth, and explain why most modern large-scale Transformers use pre-LN.
Rubric: pre-LN keeps a clean (normalization-free) residual stream, improving gradient flow and stabilizing very deep stacks; post-LN can require careful warmup/learning-rate tuning to avoid divergence.

11.2 Topic 8.2: Encoder-Decoder vs. Decoder-Only Architectures

Core Concepts:

- Encoder-decoder Transformers: bidirectional encoder attention over the source, causally-masked decoder attention plus cross-attention to the encoder – suited to conditional sequence-to-sequence modeling $p(y | x)$.
- Decoder-only Transformers: a single causally-masked stack modeling the joint sequence distribution $p(x_1, \dots, x_T) = \prod_t p(x_t | x_{<t})$ directly.
- Statistical framing of the architectural choice: is the task naturally conditional (translation, summarization) or naturally a single joint autoregressive stream (open-ended generation)?

Learning Objectives:

- Explain the causal masking mechanism and why it is necessary for valid autoregressive factorization.
- Derive the factorization $p(x_{1:T}) = \prod_t p(x_t | x_{<t})$ that decoder-only models exploit.
- Evaluate which architecture (encoder-decoder vs. decoder-only) is the better statistical fit for a given task, given its conditional structure.

Example Questions:

1. (*Basic*) Write the causal mask condition for decoder self-attention (which key positions a query at position t may attend to) and explain why omitting it would break the autoregressive factorization at training time.

Answer: position t may attend only to positions $\leq t$; without masking, the model could “see the future” token it is trying to predict, invalidating the likelihood.

2. (*Intermediate*) Explain why machine translation was historically modeled with encoder-decoder architectures rather than a single decoder-only stream.

Rubric: references the natural conditional structure $p(y | x)$ with full bidirectional access to the (already fully known) source sequence via the encoder.

3. (*Advanced*) Modern large language models have largely converged on decoder-only architectures even for tasks that are naturally conditional (e.g., translation via prompting). Give a statistical or practical argument for why a single unified autoregressive objective might be preferred despite this apparent mismatch.

Rubric: credit for arguments re: architectural simplicity/scalability, or re: a single pre-training objective transferring to many conditional tasks via prompting rather than needing task-specific architectures.

12 Module 9: Large Language Models – Pretraining, Scaling, In-Context Learning

12.1 Topic 9.1: Pretraining Objectives and Scaling Laws

Core Concepts:

- Autoregressive pretraining as large-scale MLE: maximize $\sum_t \log p_\theta(x_t | x_{<t})$ over massive text corpora.
- Empirical scaling laws: test loss following an approximate power law in model size, data size, and compute, e.g. $L(N) \approx (N_c/N)^{\alpha_N}$ holding other factors fixed.
- Compute-optimal allocation (Chinchilla-style findings, referenced generically): for a fixed compute budget, there is a statistically-fit optimal split between model size and data size.

Learning Objectives:

- Explain autoregressive pretraining as an instance of the MLE framing introduced in Topic 1.1, now applied at scale.

- Derive/interpret a power-law fit from log-log loss-vs-compute data, including how to read off the scaling exponent.
- Evaluate what compute-optimal scaling implies about jointly choosing model size and dataset size, rather than scaling one while holding the other fixed.

Common Difficulty & Mitigation: Scaling laws are often presented as an empirical curiosity rather than what they are: a statistical fit (with uncertainty) extrapolated well beyond the regime it was fit on. Mitigation: have students fit a power law to a small provided loss-vs-compute dataset themselves (log-log linear regression) so they experience firsthand both the fit and its extrapolation uncertainty, rather than taking a headline exponent on faith.

Example Questions:

1. (*Basic*) Given $L(N) = (N_c/N)^{\alpha_N}$, show how taking logs turns this into a linear relationship, and explain how you would estimate α_N from data.
Answer: $\log L = \alpha_N \log N_c - \alpha_N \log N$; slope of $\log L$ vs. $\log N$ gives $-\alpha_N$, estimable by linear regression.
2. (*Intermediate*) Explain, in one or two sentences, what “compute-optimal” means in the scaling-laws literature, and why simply making a model as large as possible for a fixed compute budget is not the loss-optimal choice.
Rubric: correctly identifies that both model size and training-data size trade off against each other under a fixed compute budget, and that some balance minimizes loss.
3. (*Advanced*) Scaling laws are fit on a certain range of model/data sizes and then used to extrapolate to much larger scales. What statistical risks does this extrapolation carry, and how would you empirically check whether a scaling law still holds at a new, larger scale?
Rubric: credit for identifying extrapolation risk (fit region vs. prediction region), and for proposing a check such as holding out a scale point or fitting on a subset and validating on a larger held-out point.

12.2 Topic 9.2: In-Context Learning and Fine-Tuning

Core Concepts:

- In-context learning (ICL): a pretrained model adapts to a new task from examples in its prompt alone, without gradient updates.
- One statistical account of ICL: the model performs something resembling implicit Bayesian inference over latent “tasks” seen during pretraining, updating a posterior over which task is being demonstrated as it reads the prompt.
- Fine-tuning and parameter-efficient adaptation: LoRA as a low-rank statistical constraint on the weight update, $\Delta W = BA$ with B, A low-rank, reducing the effective number of adapted parameters.

Learning Objectives:

- Explain the phenomenon of in-context learning and how it differs from standard supervised fine-tuning.
- Evaluate the implicit-Bayesian-inference account of ICL, including what evidence supports it and what remains unexplained.
- Derive how LoRA’s low-rank factorization reduces the number of trainable parameters relative to full fine-tuning, and state the statistical assumption behind why a low-rank update suffices.

Common Difficulty & Mitigation: In-context learning is an area of active research with competing theoretical accounts, and students often expect a single settled explanation. Mitigation: explicitly frame this topic as “theories under active debate,” present the implicit-Bayesian-inference account and induction-heads mechanistic account side by side, and have students identify what

observation would favor one account over the other rather than asking them to pick the “correct” one.

Example Questions:

1. (*Basic*) Contrast in-context learning and standard fine-tuning in terms of whether model parameters θ change.
Answer: ICL: θ fixed, task inferred from the prompt at inference time; fine-tuning: θ updated via gradient steps on task-specific data.
2. (*Intermediate*) Under the implicit-Bayesian-inference account of ICL, describe (informally) what role the prompt examples play and what quantity the model is implicitly updating as it reads more examples.
Rubric: prompt examples act as evidence; the model implicitly updates something like a posterior over latent tasks/concepts learned during pretraining.
3. (*Advanced*) LoRA constrains ΔW to be low-rank. State the implicit statistical assumption this makes about the task-adaptation update, and describe a scenario (task type) where this assumption might be a poor fit.
Rubric: assumption: the useful update direction lies in a low-dimensional subspace; poor fit example: a task requiring broad, high-rank changes across many unrelated directions in weight space (credit for any reasonable such example).

13 Module 10: Latent-Variable Generative Models

13.1 Topic 10.1: Variational Autoencoders

Core Concepts:

- Latent-variable generative model: $p_\theta(x) = \int p_\theta(x | z) p(z) dz$, generally intractable to compute or maximize directly.
- Evidence lower bound (ELBO): $\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - D_{KL}(q_\phi(z | x) \| p(z))$, replacing the intractable marginal likelihood with a tractable surrogate.
- Reparameterization trick: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, making the ELBO differentiable w.r.t. ϕ via a pathwise gradient estimator.
- Amortized inference: a single encoder network $q_\phi(z | x)$ shared across all data points, rather than per-example variational parameters.

Learning Objectives:

- Derive the ELBO from Jensen’s inequality (or the KL-divergence-to-the-true-posterior decomposition) starting from $\log p_\theta(x)$.
- Explain why the reparameterization trick is necessary to backpropagate through a sampling operation.
- Evaluate the role of amortized inference in making VAE training scale to large datasets.

Common Difficulty & Mitigation: The ELBO derivation and the reparameterization trick are classic hard points: the former requires comfort with Jensen’s inequality and KL divergence simultaneously, the latter requires seeing why $\nabla_\phi \mathbb{E}_{z \sim q_\phi}[\cdot]$ is not directly differentiable. Mitigation: derive the ELBO two ways – Jensen’s inequality, and the exact decomposition

$$\begin{aligned} \log p_\theta(x) &= \text{ELBO} \\ &\quad + D_{KL}(q_\phi(z|x) \| p_\theta(z|x)) \end{aligned}$$

so students see both why it’s a bound and how tight it is; introduce the reparameterization trick with a 1-D scalar Gaussian example before the full multivariate case.

Example Questions:

1. (*Basic*) Derive the ELBO from $\log p_\theta(x)$ using Jensen’s inequality applied to

$$\log \mathbb{E}_{q_\phi(z|x)}[p_\theta(x, z)/q_\phi(z|x)].$$

Answer: standard Jensen’s-inequality derivation ending in the ELBO expression.

2. (*Intermediate*) Explain why $\nabla_\phi \mathbb{E}_{z \sim q_\phi(z|x)}[f(z)]$ cannot be directly estimated by sampling z and differentiating, and how the reparameterization $z = \mu_\phi + \sigma_\phi \odot \epsilon$ fixes this.

Rubric: identifies that the sampling distribution itself depends on ϕ , so naive Monte Carlo differentiation is invalid; reparameterization moves the ϕ -dependence into a deterministic function of a fixed-distribution noise variable.

3. (*Advanced*) The gap between $\log p_\theta(x)$ and the ELBO equals $D_{KL}(q_\phi(z|x) \parallel p_\theta(z|x))$. Explain what this means for how well a VAE trained by maximizing the ELBO approximates true maximum likelihood, and what would make this gap large.

Rubric: correctly states that maximizing the ELBO is equivalent to maximum likelihood only if the variational family is expressive enough for q_ϕ to match the true posterior; a restrictive q_ϕ (e.g., diagonal Gaussian) leaves a persistent gap.

13.2 Topic 10.2: Normalizing Flows

Core Concepts:

- Change-of-variables formula: $\log p_X(x) = \log p_Z(f^{-1}(x)) + \log |\det J_{f^{-1}}(x)|$, giving *exact* likelihood when f is invertible.
- Architectural constraint: layers must be invertible with an efficiently computable Jacobian determinant (e.g., coupling layers with triangular Jacobians).
- Tradeoff vs. VAEs: flows give exact likelihoods but require restrictive architectures; VAEs allow flexible architectures but only a likelihood lower bound.

Learning Objectives:

- Derive the change-of-variables formula for a scalar invertible transform and state its multivariate generalization.
- Explain why coupling-layer architectures are designed specifically to make the Jacobian determinant cheap to compute (triangular structure).
- Evaluate the exact-likelihood-vs-architectural-flexibility tradeoff between normalizing flows and VAEs.

Common Difficulty & Mitigation: The Jacobian-determinant term is often where students lose the thread, particularly why architectures are constrained to make it tractable at all. Mitigation: start from the 1-D case (where the “Jacobian” is just $|f^{-1'}(x)|$, a single scalar derivative) and only then generalize to the multivariate determinant, explicitly connecting triangular-Jacobian coupling layers back to the 1-D intuition (determinant of a triangular matrix is just the product of the diagonal).

Example Questions:

1. (*Basic*) For a scalar invertible, monotonic f , derive $p_X(x) = p_Z(f^{-1}(x)) |(f^{-1})'(x)|$ from first principles using the CDF transformation.

Answer: standard 1-D change-of-variables derivation via differentiating the transformed CDF.

2. (*Intermediate*) Explain why coupling-layer architectures split the input into two parts and transform only one conditioned on the other, in terms of what this buys computationally for the Jacobian determinant.

Rubric: identifies that this construction yields a triangular Jacobian, whose determinant is just the product of diagonal entries – $O(d)$ instead of $O(d^3)$ for a general determinant.

3. (*Advanced*) Compare normalizing flows and VAEs on: (a) whether the likelihood computed is exact or a bound, and (b) architectural flexibility. Give one modeling scenario where each would be preferred.

Rubric: credit for correctly stating (a) and (b) and for any reasonable scenario per model (e.g., flows when exact density/likelihood evaluation is required; VAEs when a compact, semantically useful latent space matters more than exact likelihood).

14 Module 11: Adversarial and Score-Based Generative Models

14.1 Topic 11.1: Generative Adversarial Networks

Core Concepts:

- Minimax game: $\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$, an implicit generative model with no explicit likelihood.
- At the theoretical optimum (optimal D), the generator's objective reduces to minimizing the Jensen–Shannon divergence between p_{data} and p_G – a statistical-distance interpretation of adversarial training.
- Practical training instability: non-convex-non-concave games need not converge to the theoretical equilibrium; mode collapse as a specific, common failure mode.
- Alternative statistical distances (e.g., Wasserstein distance in WGAN) proposed to address vanishing gradients and instability of the JS-divergence formulation.

Learning Objectives:

- Derive, at the optimal discriminator, that the generator's objective corresponds to minimizing (a shifted/scaled) Jensen–Shannon divergence.
- Explain mode collapse as a statistical failure mode and connect it to the asymmetry of the JS/KL divergences involved.
- Evaluate why swapping the statistical distance (e.g., to Wasserstein) can improve training stability.

Common Difficulty & Mitigation: GANs are often taught as a training recipe without the statistical-distance derivation, leaving students unable to reason about *why* training is unstable or what a proposed fix (e.g., WGAN) is actually fixing. Mitigation: derive the optimal discriminator $D^*(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_G(x))$ explicitly, substitute back to reveal the JS-divergence objective, and only then discuss instability/mode collapse as consequences of that specific divergence's properties.

Example Questions:

1. (*Basic*) Starting from the minimax objective, derive the optimal discriminator $D^*(x)$ for a fixed generator G .
Answer: $D^*(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_G(x))$, from setting the pointwise derivative of the integrand to zero.
2. (*Intermediate*) Substituting D^* back into the objective, show (at a sketch level) that the generator's objective becomes a monotonic function of the Jensen–Shannon divergence between p_{data} and p_G .
Rubric: correctly identifies the resulting expression as $2 \text{JSD}(p_{\text{data}} \| p_G) - \log 4$ (exact constants not required, correct divergence identification is).
3. (*Advanced*) Explain mode collapse as a statistical phenomenon: why might a generator prefer to model only a subset of the modes of p_{data} rather than the full distribution, given the minimax objective?

Rubric: credit for connecting this to the generator's incentive structure (fooling the current

discriminator locally) rather than any explicit penalty for missing modes, distinguishing it from, e.g., a KL-divergence-driven mode-covering estimator.

14.2 Topic 11.2: Diffusion and Score-Based Generative Models

Core Concepts:

- Score function $\nabla_x \log p(x)$ as the object of interest; score matching estimates it without needing the (intractable) normalizing constant of $p(x)$.
- Forward diffusion process: progressively adding Gaussian noise to data until it approaches a known prior (e.g., standard Gaussian); reverse process: learning to denoise step by step, guided by the estimated score.
- Langevin dynamics as a sampling procedure using the score: $x_{t+1} = x_t + \frac{\epsilon}{2} \nabla_x \log p(x_t) + \sqrt{\epsilon} \eta_t$, $\eta_t \sim \mathcal{N}(0, I)$.
- Connection to stochastic differential equations (SDEs): the discrete forward/reverse diffusion steps as a discretization of a continuous-time SDE, linking diffusion models to statistical physics (Langevin/Fokker–Planck).

Learning Objectives:

- Explain why score matching sidesteps the intractable normalizing constant that makes direct likelihood-based training of unnormalized models hard.
- Derive the Langevin-dynamics sampling update and explain the role of each term (drift via the score, injected noise).
- Evaluate the forward/reverse diffusion process as a statistically principled way to convert sampling from a complex p_{data} into a sequence of easier denoising problems.

Common Difficulty & Mitigation: This topic asks students to absorb unfamiliar machinery (score functions, SDEs, Langevin dynamics) all at once, often without prior exposure to stochastic calculus. Mitigation: introduce the score function and score matching purely in a static (non-diffusion) setting first, with a simple 1-D or 2-D toy density; only once the score concept itself is solid, layer on the forward-noise/reverse-denoise diffusion process and its SDE interpretation as an extension, not a prerequisite, of the core score-matching idea.

Example Questions:

1. (*Basic*) Write the score function $\nabla_x \log p(x)$ for a 1-D Gaussian $p(x) = \mathcal{N}(x; \mu, \sigma^2)$ and interpret its sign/direction.
Answer: $\nabla_x \log p(x) = -(x - \mu)/\sigma^2$; points back toward the mean, with magnitude proportional to distance from it.
2. (*Intermediate*) Explain why estimating $\nabla_x \log p(x)$ via score matching avoids needing to know the normalizing constant Z of an unnormalized density $p(x) = \tilde{p}(x)/Z$.
Rubric: correctly notes $\nabla_x \log p(x) = \nabla_x \log \tilde{p}(x)$ since $\nabla_x \log Z = 0$ (Z does not depend on x).
3. (*Advanced*) Describe, at a conceptual level, how the forward diffusion (data $\rightarrow$ noise) and reverse diffusion (noise $\rightarrow$ data) processes can be understood as time-reversed versions of the same stochastic process, and why this framing lets a single learned score function support sampling at many noise levels.
Rubric: credit for connecting to the SDE/time-reversal framing generically (exact Fokker–Planck derivation not required) and for explaining that a noise-conditional score model generalizes across the whole forward trajectory rather than needing a separate model per noise level.

15 Module 12: Self-Supervised and Contrastive Representation Learning

15.1 Topic 12.1: Contrastive Learning

Core Concepts:

- Contrastive objective: pull representations of augmented “positive pairs” together, push representations of unrelated “negative” examples apart, without requiring labels.
- InfoNCE loss as a lower bound on mutual information between two views of the data: maximizing it is (approximately) maximizing a mutual-information estimate.
- Choice and statistics of negative sampling (batch size, hard-negative mining) as a key practical/statistical design decision affecting the tightness of the MI bound.

Learning Objectives:

- Explain the contrastive learning objective and how positive/negative pairs are constructed from unlabeled data.
- Derive (at a sketch level) why the InfoNCE loss lower-bounds mutual information between two augmented views.
- Evaluate how the number and quality of negative samples affects the statistical tightness of the mutual-information bound being optimized.

Example Questions:

1. (*Basic*) Describe how positive and negative pairs are constructed for a contrastive image representation learning method using data augmentation alone (no labels).
Answer: two augmented views of the same image form a positive pair; views from different images in the batch form negative pairs.
2. (*Intermediate*) State the InfoNCE loss and explain, informally, why maximizing it corresponds to maximizing a lower bound on mutual information between the two views.
Rubric: correctly writes a softmax-style loss over one positive and K negatives; explains the bound tightens as the number of negatives increases.
3. (*Advanced*) Explain why increasing the number of negative samples per positive pair tends to tighten the InfoNCE mutual-information bound, and identify one practical cost of doing so.
Rubric: references the bound’s dependence on $\log K$ (number of negatives); practical cost: larger batch sizes / more compute and memory per step.

15.2 Topic 12.2: Masked and Predictive Self-Supervision

Core Concepts:

- Masked modeling objectives (e.g., masking tokens/patches and predicting them from context) as a form of pseudolikelihood estimation: modeling each masked element’s conditional distribution given the rest.
- Relationship to the autoregressive MLE framing from Module 9: masked modeling trades the strict left-to-right factorization for bidirectional context, at the cost of no longer maximizing an exact joint likelihood.
- Design choices: masking ratio and masking strategy as statistical knobs controlling the difficulty/informativeness of the self-supervised task.

Learning Objectives:

- Explain masked modeling as an instance of pseudolikelihood estimation, contrasting it with exact joint MLE.

- Compare masked (bidirectional) and autoregressive (left-to-right) pretraining objectives in terms of what each does and does not directly optimize.
- Evaluate how the masking ratio affects the statistical difficulty and informativeness of the pretraining task.

Example Questions:

1. (*Basic*) Write the masked-modeling training objective for predicting a masked token x_i given the unmasked context, and compare its form to the autoregressive objective from Module 9.
Answer: $-\log p_\theta(x_i | x_{\setminus i})$ summed over masked positions, versus $-\log p_\theta(x_t | x_{<t})$ for autoregressive modeling – both conditional log-likelihoods, but conditioned on different information sets.
2. (*Intermediate*) Explain why masked modeling is described as optimizing a pseudolikelihood rather than the exact joint likelihood $p(x_1, \dots, x_T)$.
Rubric: identifies that summing independent conditional log-likelihoods over masked positions does not correspond to a valid factorization of the true joint density (the conditionals are not consistent with any single joint in general).
3. (*Advanced*) How would you expect model behavior to change as the masking ratio increases from very low (e.g., 5%) to very high (e.g., 80%)? Frame your answer in terms of task difficulty and the amount of context available per prediction.
Rubric: credit for reasoning that very low ratios make the task too easy (little signal, mostly copying context) while very high ratios remove too much context (task becomes closer to unconditional generation, potentially harder to learn from efficiently); no single correct optimal ratio required.

16 Module 13: Graph Neural Networks and Structured Data

16.1 Topic 13.1: Message Passing as a Statistical Belief-Propagation Analogy

Core Concepts:

- Message-passing update: $h_v^{(k)} = \text{UPDATE}(h_v^{(k-1)}, \text{AGG}(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}))$, iteratively refining node representations using neighborhood information.
- Permutation invariance/equivariance as the graph analogue of the translation-equivariance assumption behind convolution (Module 5): the aggregation function must not depend on an arbitrary ordering of neighbors.
- Loose analogy (and important disanalogy) to belief propagation in probabilistic graphical models: both pass local messages along graph structure, but GNN messages are learned, feedforward, and not generally tied to exact marginal-probability computation.

Learning Objectives:

- Explain why the AGG function in message passing must be permutation-invariant (e.g., sum, mean, max) rather than, e.g., concatenation in a fixed order.
- Derive how stacking k message-passing layers expands a node’s effective receptive field to its k -hop neighborhood, analogous to receptive field growth in CNNs (Module 5).
- Evaluate the analogy and disanalogy between GNN message passing and belief propagation in graphical models.

Example Questions:

1. (*Basic*) Explain why using a fixed-order concatenation of neighbor features as the AGG function would break permutation invariance, and give a valid alternative.
Answer: concatenation depends on an arbitrary neighbor ordering that does not exist canonically in a graph; sum, mean, or max are valid permutation-invariant alternatives.

2. (*Intermediate*) Using the receptive-field-growth argument from Module 5 (Topic 5.1) as a template, explain why k layers of message passing let a node’s representation depend on its k -hop neighborhood.

Rubric: correctly draws the direct analogy to CNN receptive field growth with depth.

3. (*Advanced*) Belief propagation computes exact marginals on tree-structured graphical models under specific conditions. Explain why GNN message passing does not inherit this exactness guarantee, even though the update rule looks structurally similar.

Rubric: credit for identifying that GNN messages are learned, generic feedforward functions rather than exact probabilistic updates tied to a specified joint distribution, and that most real graphs are not tree-structured.

16.2 Topic 13.2: Spectral GNNs and Expressiveness Limits

Core Concepts:

- Spectral graph convolution via the eigendecomposition of the graph Laplacian, as an alternative formulation to spatial message passing.
- Weisfeiler-Leman (WL) graph isomorphism test as an upper bound on the expressive power of standard message-passing GNNs: two graphs indistinguishable by 1-WL are also indistinguishable by standard message passing.
- Over-smoothing: as message-passing depth increases, node representations across the graph statistically converge toward each other, degrading discriminative power – a graph-specific analogue of the vanishing-signal problems seen in very deep networks (Modules 5, 6).

Learning Objectives:

- Explain the spectral-convolution formulation and how it relates to the graph Laplacian’s eigendecomposition.
- Evaluate the claim that standard message-passing GNNs are no more powerful than the 1-WL test, and what this implies about their limits (e.g., inability to distinguish certain regular graphs).
- Explain over-smoothing as a statistical convergence phenomenon and connect it to analogous depth-related issues encountered earlier in the course (vanishing gradients, degradation).

Common Difficulty & Mitigation: The WL-test expressiveness bound is often presented as an abstract theoretical result disconnected from practice, and over-smoothing can seem like an unrelated engineering nuisance rather than the same depth-degradation theme seen throughout the course. Mitigation: show a concrete pair of simple graphs (e.g., two graphs 1-WL cannot distinguish) that a standard GNN provably cannot tell apart, making the expressiveness limit tangible; explicitly connect over-smoothing back to the vanishing-gradient (Module 6) and degradation (Module 5) discussions as the same depth-related theme recurring in a new setting.

Example Questions:

1. (*Basic*) State, informally, what the 1-WL graph isomorphism test does (iteratively relabeling nodes by aggregating neighbor labels) and why this procedure resembles a single message-passing layer.
Answer: both iteratively update a node’s label/representation as a function of its neighbors’ labels/representations.
2. (*Intermediate*) Explain what it means for standard message-passing GNNs to be “no more powerful than 1-WL,” and give a concrete implication for what such a GNN can and cannot distinguish.

Rubric: correctly states that any two graphs indistinguishable by 1-WL will receive identical GNN representations under sum-aggregation message passing, e.g., certain pairs of regular

graphs.

3. (*Advanced*) Explain over-smoothing as node representations statistically converging with increasing message-passing depth, and connect this explicitly to one other depth-related phenomenon covered earlier in the course.

Rubric: full credit requires an explicit connection to vanishing gradients (Module 6) or the plain-network degradation problem (Module 5), not just a description of over-smoothing in isolation.

17 Module 14: Uncertainty, Frontiers, and Synthesis

17.1 Topic 14.1: Bayesian Deep Learning and Uncertainty Quantification

Core Concepts:

- Bayesian neural networks: placing a prior over weights and seeking (an approximation to) the posterior $p(\theta \mid \mathcal{D})$, rather than a single point estimate.
- Practical approximations: MC-dropout (using dropout at test time as approximate posterior sampling, connecting back to Topic 4.1) and deep ensembles (training multiple independently-initialized networks) as two common ways to approximate predictive uncertainty.
- Calibration: whether a model’s predicted probabilities match empirical outcome frequencies, and why standard deep networks are often overconfident/miscalibrated.

Learning Objectives:

- Explain the conceptual difference between a point-estimate network and a Bayesian neural network in terms of what each represents about parameter uncertainty.
- Compare MC-dropout and deep ensembles as two practical approximations to Bayesian predictive uncertainty, referencing the Topic 4.1 dropout-as-approximate-inference connection.
- Evaluate model calibration using a reliability diagram and explain why high accuracy does not imply good calibration.

Example Questions:

1. (*Basic*) Explain the conceptual difference between training a single point-estimate network and approximating a full posterior $p(\theta \mid \mathcal{D})$ over network weights.
Answer: a point estimate gives one set of weights (e.g., the MAP/MLE solution); a posterior gives a distribution over plausible weight settings, enabling predictive uncertainty by marginalizing over it.
2. (*Intermediate*) Explain how MC-dropout at test time can be interpreted as approximately sampling from a posterior predictive distribution, connecting this back to the dropout-as-approximate-Bayesian-inference idea from Module 4.
Rubric: correctly references running multiple stochastic forward passes with dropout active at test time and treating the resulting output distribution as an approximate predictive distribution.
3. (*Advanced*) A model achieves 95% test accuracy but its predicted confidence on incorrect predictions averages 90%. Explain what this indicates about calibration and propose one way to diagnose this more rigorously than a single summary statistic.
Rubric: identifies overconfidence/miscalibration; proposes a reliability diagram or expected calibration error (ECE) as a more rigorous diagnostic.

17.2 Topic 14.2: Frontiers and Course Synthesis (Buffer Session)

Core Concepts:

- Brief survey of modern efficient/emerging architectures, framed through the course’s recurring statistical lens: state-space sequence models (e.g., S4/Mamba-style architectures) as structured linear dynamical systems with learned, input-dependent transitions; mixture-of-experts layers as a learned statistical mixture model over sub-networks, with a gating function acting as a mixture-weight estimator.
- Course-wide synthesis: revisit the recurring throughline – every architecture in this course encodes a specific statistical assumption (stationarity, conditional independence structure, low-rank structure, latent-variable structure) chosen to trade bias for variance in a particular data domain.
- This session is the semester’s designated buffer/flex session (per the intake discussion): if the semester runs 13 weeks instead of 14, this session’s content is the first to compress or drop; if the semester runs the full 14 weeks, it serves as open time for review, deeper Q&A on Project 5, or the frontier topics above.

Learning Objectives:

- Explain state-space sequence models and mixture-of-experts layers using the same statistical-inductive-bias vocabulary developed throughout the course.
- Synthesize, across at least three modules, the recurring pattern of “architecture as encoded statistical assumption” that has structured the course.

Example Questions:

1. (*Basic*) Explain, in one or two sentences, why a mixture-of-experts layer can be described as a learned statistical mixture model, and identify which component plays the role of the mixture weights.
Answer: the gating network’s output plays the role of (input-dependent) mixture weights over the expert sub-networks.
2. (*Intermediate*) Modern state-space sequence models are sometimes described as combining ideas from RNNs (Module 6) and convolution (Module 5). Explain, at a conceptual level, what each contributes.
Rubric: credit for identifying the RNN-like sequential/recurrent state update alongside a convolution-like parallelizable/structured computation for training efficiency.
3. (*Advanced*) Choose three architectures from earlier in the course (e.g., CNNs, Transformers, VAEs) and, for each, state the specific statistical assumption it encodes and what kind of data that assumption suits. Then argue whether a fourth architecture of your choosing (from the course or the frontier topics above) fits the same “bias-for-variance trade via a structural assumption” pattern, or breaks it.
Rubric: full credit requires correct, specific (not generic) statistical assumptions for the three chosen architectures and a substantive, non-trivial argument for the fourth – either fitting or genuinely breaking the pattern with justification.